

Speedforce Marathon 0001

07 July 2025

Help Percy Alexander MacMahon

Input file: standard input

Output file: standard output

Time limit: 1 seconds

Memory limit: 512 megabytes

Srinivasa Ramanujan is one of the greatest mathematicians in the world, known for his groundbreaking contributions in number theory. He was the youngest fellow at the Royal Society of England. Ramanujan had an intuitive grasp of numbers and formulas; in his dreams, the goddess **Namagiri** supposedly inscribed equations on his neck, often producing results that seemed to come out of nowhere.

One of the problems he posed to his colleague G.H. Hardy was related to partitions - specifically, finding a formula to count the number of ways a positive integer can be expressed as a sum of positive integers, regardless of the order of the summands. This problem led to the development of the famous **Hardy-Ramanujan asymptotic formula** for the partition function, which provides an approximation for the number of partitions of a given integer. Their collaboration on this and other mathematical puzzles left an indelible mark on the field of number theory.

$$p(n) \sim \frac{1}{4n\sqrt{3}} e^{\pi\sqrt{n/3}}$$

For testing the Hardy-Ramanujan asymptotic formula, a mathematician named Percy Alexander MacMahon was hired. He had to count $p(200)$. At that time, there were no computers or programming languages available. Now, let's help Percy Alexander MacMahon count the partition of 200 or any number.

In mathematics, a partition of a positive integer n is a way of expressing n as a sum of positive integers.

If $n = 4$, then $p(4) = 5$:

1. 4
2. $3 + 1$
3. $2 + 2$
4. $2 + 1 + 1$
5. $1 + 1 + 1 + 1$

Input

The first line of input contains one integer t ($1 \leq t \leq 10^4$) representing the number of test cases. After that, t test cases follow. Each of them is presented as follows.

The first line contains one integer n ($1 \leq n \leq 3000$).

Output

For each test case, print a single integer $p(n)$ — the answer to the problem modulo $1000000007(10^9 + 7)$.

Example

Standard Input	Standard Output
4	5
4	7
5	11
6	999001584
200	

Explanation

1st test case explained in the statement.

In the second test: $p(5) = 7$

$p(5) = [(5), (4+1), (3+2), (3+1+1), (2+1+1+1), (2+1+2), (1+1+1+1+1)]$

You can represent 5 as the sum in 7 possible ways.

Note

The following problem is **NP-hard** (Non-deterministic Polynomial-time hard).

Sourav's Sequence

Input file: standard input

Output file: standard output

Time limit: 2 seconds

Memory limit: 512 megabytes

Our **Sourav** is curious about lot of thing's. He is fond of finding new sequences and implementing them in code. Recently, he learned about recursion, so he started **counting the number of '1's in the binary representation of a number**. Like, $\text{OneBitCount}(5) = 2$; $5 = (101)_2$.

Sourav's code in c/c++:

```
int OneBitCount(int m){
    return ((m == 0)?0:((m&1) + OneBitCount(m >> 1)));
}
```

Now, he thinks that if he increases the value of m one by one, it will form a sequence.

Like, n is the number then the sequence will be : $\text{OneBitCount}(m), \text{OneBitCount}(m+1), \text{OneBitCount}(m+2), \dots$

If $m = 5$ then the sequence: 2, 2, 3,.....

Now, he is curious about whether every sequence can form this kind of number m ?

You are given a sequence of n integers $(p_1, p_2, p_3, \dots, p_{n-1})$. Your task is to determine whether there exists a non-negative integer m such that $(\text{OneBitCount}(m), \text{OneBitCount}(m+1), \text{OneBitCount}(m+2), \dots, \text{OneBitCount}(m+n-1))$ is equal to $(p_1, p_2, p_3, \dots, p_{n-1})$.

Help Sourav figure out whether the sequence exists or not.

Input

The first line of input contains one integer t ($1 \leq t \leq 10^4$) representing the number of test cases. After that, t test cases follow. Each of them is presented as follows.

The first line contains one integer n ($1 \leq n \leq 10000$) the length of the array. The second line contains n integers $p_1, p_2, \dots, p_{n-1}$ ($0 \leq p_i \leq 63$)

Output

For each test case, If such a sequence exists, output **YES**; otherwise, output **NO**.

Standard Input	Standard Output
3	YES
2	NO
2 2	YES
2	
1 3	
3	
2 2 3	

Explanation

In the first test case the sequence exists:

Because, 2, 2 is equal to $\text{OneBitCount}(5) = 2$, $\text{OneBitCount}(5+1) = 2$.

In the second test case the sequence doesn't exist:

Because, 1 3 is not equal to $\text{OneBitCount}(1) = 1$, $\text{oneBitCount}(1+1) = 1$

In the third test case the sequence exists:

Because, 2 2 3 is equal to $\text{OneBitCount}(5) = 2$, $\text{OneBitCount}(5+1) = 2$, $\text{OneBitCount}(5+2) = 3$

Note

Here, m doesn't exceed $2^{63} - 1$. where m is the parameter of $\text{OneBitCount}()$ function.

Mutual Friends in NUBTK

Input file: standard input

Output file: standard output

Time limit: 4 seconds

Memory limit: 512 megabytes

At NUBTK (Northern University of Business and Technology Khulna), every semester, students form numerous groups based on shared interests or other factors.

Each student in the group is connected to others through either a **mutual connection** or a **direct connection**, creating a network of mutual connections. The university authorities have tasked *Shisir vai and Rony vai* with identifying the number of distinct groups within the student's. If a student is not connected to any group, we assume they have individual interests, and each such student is considered to form a group of their own.

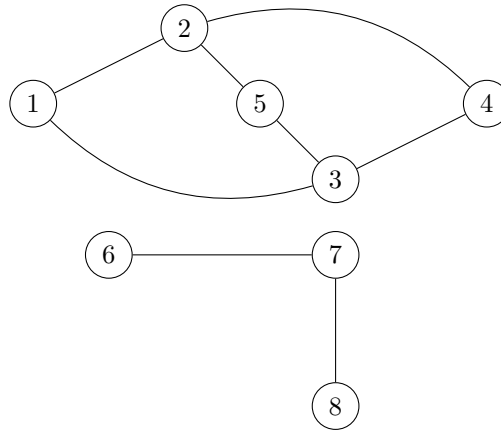


Fig 1.1 : Connection between the student's

Here, S_1 is directly connected to S_2 , S_3 , and S_5 , and mutually connected to S_4 . Thus, S_1 , S_2 , S_3 , S_4 , and S_5 are all connected to each other either by direct connection or mutual connection.

In the same way, S_6 , S_7 , and S_8 are connected to each other. However, S_1 , S_2 , S_3 , S_4 , and S_5 are not connected to S_6 , S_7 , and S_8 .

In this semester, there are 2 different groups.

Input

The first line of input contains one integer t ($1 \leq t \leq 10^4$) representing the number of test cases. After that, t test cases follow. Each of them is presented as follows.

The first line contains two integer v ($1 \leq v \leq 1000$) and e ($0 \leq e \leq 1000$) the number of vertex and the number of connection between the vertex or edge. The next e line will contain two integers, n_i and n_j , representing the connection between the nodes.

Output

For each test case, output the number of different group.

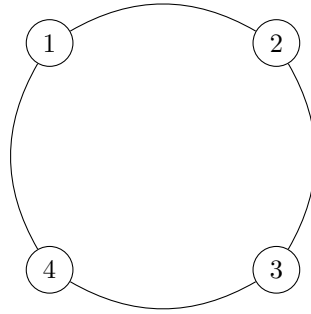
Standard Input	Standard Output
2	4
8 9	2
1 2	
1 3	
2 4	
2 5	
3 5	
3 4	
5 3	
6 7	
7 8	
4 4	
1 2	
2 3	
3 4	
4 1	

Explanation

First test case explain in the statement.

Second test case:

Here, $v = 4$ and $e = 4$



In this example, there is only one group.

Note

Connection between students is either mutual or direct.

Custom Integer Data Type

Input file: standard input

Output file: standard output

Time limit: 1 seconds

Memory limit: 512 megabytes

We all know that classical computers only understand machine code in the form of 0s and 1s. To interact with a computer, we first need to convert our instructions or data into binary representation. In computer programming, a data type is a classification that identifies one of various types of data, such as integers, floats, doubles, and chars, each storing different kinds of information. When storing integers, we typically use binary representations of 16, 32, or 64 bits. Our friend **Twahanur** is eager to build a **custom integer data type** or n bit integer data type. so he write a code in c for 63-bit integer data type.

```
#include<stdio.h>
union IntBit{
    long long input;
    struct{
        long long obj:63;
    }nBit;
};
int main()
{
    union IntBit x;
    scanf("%lld", &(x.input));
    printf("%lld = %d\n", x.input, x.nBit.obj);
    return 0;
}
```

But he is curious about what the number x is in an n -bit's integer data type system.

Input

The first line of input contains one integer t ($1 \leq t \leq 10^4$) representing the number of test cases. After that, t test cases follow. Each of them is presented as follows.

The first line contains two integer x ($-10^{63} \leq x \leq 10^{63} - 1$) and n ($1 \leq n \leq 64$) the number and n -bit's integer data type system.

Output

For each test case, output the single integer x is in an n -bit's integer data type system.

Standard Input	Standard Output
6	1
1 2	-2
2 2	7
7 4	-8
8 4	9223372036854775807
9223372036854775807 64	-7
-7 64	

Explanation

First Test case:

$x = 1$ and $n = 2$ here, $(x)_2 = (01)_2$

here $n = 2$ so the left most bit is sign bit. For positive integer the sign bit is 0 so x is represent as 01 in 2-bit's integer data type system.

In the second test case:

$x = 2$ and $n = 2$ here, $(x)_2 = (10)_2$

When $n = 2$, the leftmost bit is 1. Then, since the compiler interprets it as a signed number, it applies two's complement: $(10)_2 \rightarrow (01)_2 + 1 \rightarrow (10)_2$. Therefore, the compiler outputs it as -2.

In the third test case:

$x = 7$ and $n = 4$, $(x)_2 = (0111)_2$. Since $n = 4$, the leftmost bit is not considered as a sign bit. Therefore, x is represented directly as 0111 in a 4-bit integer data type system.

In the 4th test case:

$x = 8$ and $n = 4$, $(x)_2 = (1000)_2$. Again, since $n = 4$, the leftmost bit is not considered as a sign bit. Therefore, x is represented directly as 1000 in a 4-bit integer data type system.

$x = 9223372036854775807$ and $n = 64$, $(x)_2$ would be a very long binary number, but essentially it's the binary representation of the positive number 9223372036854775807. Since $n = 64$, there's no need to adjust for sign bits or 2's complement representation, so the number remains unchanged.

$x = -7$ and $n = 64$, the representation would depend on the specific size of the data type. Assuming a 64-bit integer, $(x)_2$ would be $(11001)_2$. In this case, the leftmost bit is 1, indicating a negative number. So, the compiler would interpret it as such without needing to convert to 2's complement form.